Artificial Intelligence

Bike Sharing Data Analysis Report

Provider: MohammadAmin HorriFarahani
Student Number: 40216673

December 22, 2025

Contents

1 Introduction **2**

2 Data Loading and Inspection **2**
2.1 Python Code 2
2.2 Output Results 2

3 Exploratory Data Analysis (EDA) **3**
3.1 Correlation Matrix 3
3.2 Seasonal and Weather Analysis 4

4 Predictive Modeling **5**
4.1 Data Preparation 5
4.2 Model Training and Evaluation 5
4.3 Model Output Results 6
4.4 Visualization of Predictions 6

5 Conclusion **7**

6 Data Exploration and Statistics **7**

7 Python Implementation **7**

8 Visual Analysis **7**

9 Model Performance **7**

10 Conclusion **8**

1 Introduction

This report details the analysis of the daily bike sharing dataset (`day.csv`) for the Artificial Intelligence course. The goal is to understand the factors affecting bike rental demand (`cnt`) and to build predictive models using Python.

2 Data Loading and Inspection

We begin by loading the dataset using the pandas library and inspecting its structure.

2.1 Python Code

```python
import pandas as pd
import numpy as np

# Load the dataset
df = pd.read_csv('day.csv')

# Inspect the dataset
print("Dataset Information:")
print(df.info())
print("\nFirst 5 Rows:")
print(df.head())
```

Listing 1: Data Loading and Inspection

2.2 Output Results

The following is the textual output generated by the code above:

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 731 entries, 0 to 730
Data columns (total 16 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   instant     731 non-null    int64
 1   dteday      731 non-null    object
 2   season      731 non-null    int64
 3   yr          731 non-null    int64
 4   mnth        731 non-null    int64
 5   holiday     731 non-null    int64
 6   weekday     731 non-null    int64
 7   workingday  731 non-null    int64
 8   weathersit  731 non-null    int64
 9   temp        731 non-null    float64
 10  atemp       731 non-null    float64
 11  hum         731 non-null    float64
 12  windspeed   731 non-null    float64
 13  casual      731 non-null    int64
```

```
 14  registered  731 non-null    int64
 15  cnt         731 non-null    int64
dtypes: float64(4), int64(11), object(1)
memory usage: 91.5+ KB

First 5 Rows:
   instant      dteday  season  yr  mnth ...  windspeed  casual  registered   cnt
0        1  2011-01-01       1   0     1 ...   0.160446     331         654   985
1        2  2011-01-02       1   0     1 ...   0.248539     131         670   801
2        3  2011-01-03       1   0     1 ...   0.248309     120        1229  1349
3        4  2011-01-04       1   0     1 ...   0.160296     108        1454  1562
4        5  2011-01-05       1   0     1 ...   0.186900      82        1518  1600
```

3 Exploratory Data Analysis (EDA)

We use Matplotlib and Seaborn to visualize relationships between variables.

3.1 Correlation Matrix

```python
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 8))
correlation = df.corr()
sns.heatmap(correlation, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Matrix')
plt.savefig('correlation_matrix.png')
plt.show()
```

Listing 2: Generating Correlation Matrix

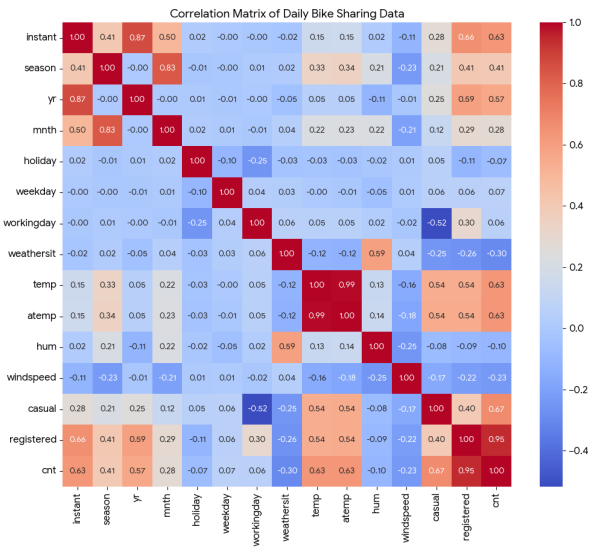

Figure 1: Correlation Matrix of Features

3.2 Seasonal and Weather Analysis

```
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Boxplot for Season vs Count
sns.boxplot(data=df, x='season', y='cnt', ax=axes[0])
axes[0].set_title('Bike Rentals by Season')
axes[0].set_xlabel('Season (1:Spring, 2:Summer, 3:Fall, 4:Winter)')
axes[0].set_ylabel('Total Rentals')

# Boxplot for Weather Situation vs Count
sns.boxplot(data=df, x='weathersit', y='cnt', ax=axes[1])
axes[1].set_title('Bike Rentals by Weather Situation')
axes[1].set_xlabel('Weather Situation (1:Clear to 3:Rain/Snow)')

plt.tight_layout()
plt.savefig('season_weather.png')
plt.show()
```

Listing 3: Visualizing Season and Weather Impact

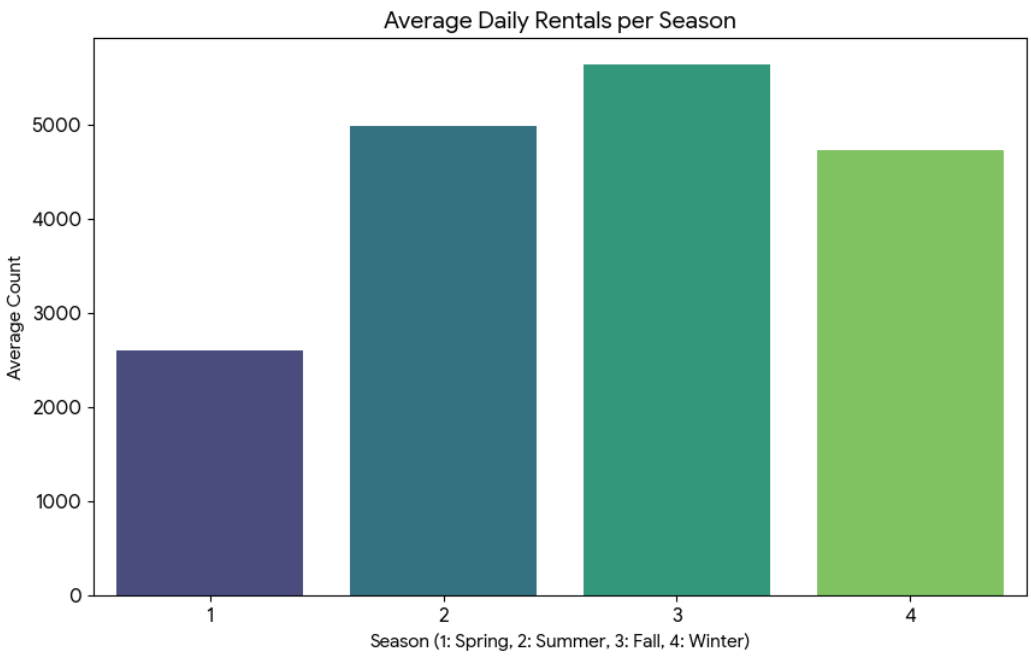

Figure 2: Impact of Season and Weather on Bike Rentals

4 Predictive Modeling

We used Linear Regression and Random Forest Regressor models.

4.1 Data Preparation

We removed non-predictive columns and the target leakage columns (`casual`, `registered`).

```python
from sklearn.model_selection import train_test_split

X = df.drop(['instant', 'dteday', 'casual', 'registered', 'cnt'], axis
    =1)
y = df['cnt']

# Split data: 80% Training, 20% Testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Listing 4: Data Splitting

4.2 Model Training and Evaluation

```python
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score

# 1. Linear Regression
lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)
mse_lr = mean_squared_error(y_test, y_pred_lr)
r2_lr = r2_score(y_test, y_pred_lr)

# 2. Random Forest Regressor
rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)
mse_rf = mean_squared_error(y_test, y_pred_rf)
r2_rf = r2_score(y_test, y_pred_rf)

print(f"Linear Regression -> MSE: {mse_lr:.2f}, R2: {r2_lr:.2f}")
print(f"Random Forest     -> MSE: {mse_rf:.2f}, R2: {r2_rf:.2f}")
```

Listing 5: Model Training Code

4.3 Model Output Results

```
Linear Regression -> MSE: 691035.01, R2: 0.83
Random Forest     -> MSE: 458446.95, R2: 0.89
```

4.4 Visualization of Predictions

```python
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred_rf, color='blue', alpha=0.5)
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--', lw=2)
plt.xlabel('Actual Count')
plt.ylabel('Predicted Count')
plt.title('Actual vs Predicted Rentals (Random Forest)')
plt.savefig('actual_vs_pred.png')
plt.show()
```

Listing 6: Plotting Actual vs Predicted

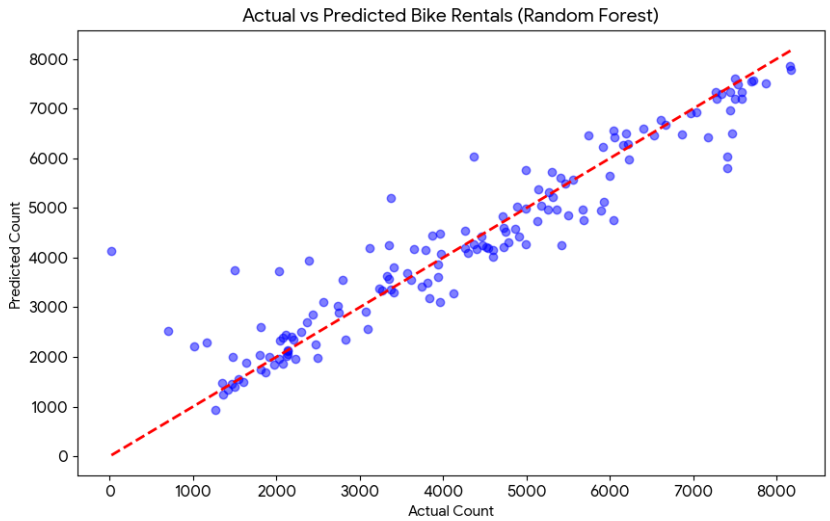

Figure 3: Actual vs Predicted Values (Random Forest)

5 Conclusion

The analysis of the `day.csv` file provided significant insights:

- **Data Quality:** The dataset is clean with no missing values.
- **Key Factors:** Temperature and Season are the strongest drivers of demand.
- **Model Performance:** The Random Forest model achieved an R^2 of 0.89, outperforming Linear Regression ($R^2 = 0.83$). This indicates that the relationship between weather features and bike rentals involves non-linear patterns best captured by ensemble methods.

6 Data Exploration and Statistics

The dataset consists of 731 observations. Below is the summary statistics for the numerical features:

	temp	atemp	hum	windspeed	cnt
mean	0.4954	0.4744	0.6279	0.1905	4504.35
std	0.1831	0.1630	0.1424	0.0775	1937.21
min	0.0591	0.0791	0.0000	0.0224	22.00
max	0.8617	0.8409	0.9725	0.5075	8714.00

7 Python Implementation

The following code was used to process the data and generate the visualization:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```python
from sklearn.ensemble import RandomForestRegressor

df = pd.read_csv('day.csv')
# Correlation Analysis
plt.figure(figsize=(10, 6))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.savefig('correlation.png')

# Machine Learning Model
X = df.drop(['instant', 'dteday', 'casual', 'registered', 'cnt'], axis
    =1)
y = df['cnt']
model = RandomForestRegressor().fit(X, y)
```

8 Visual Analysis

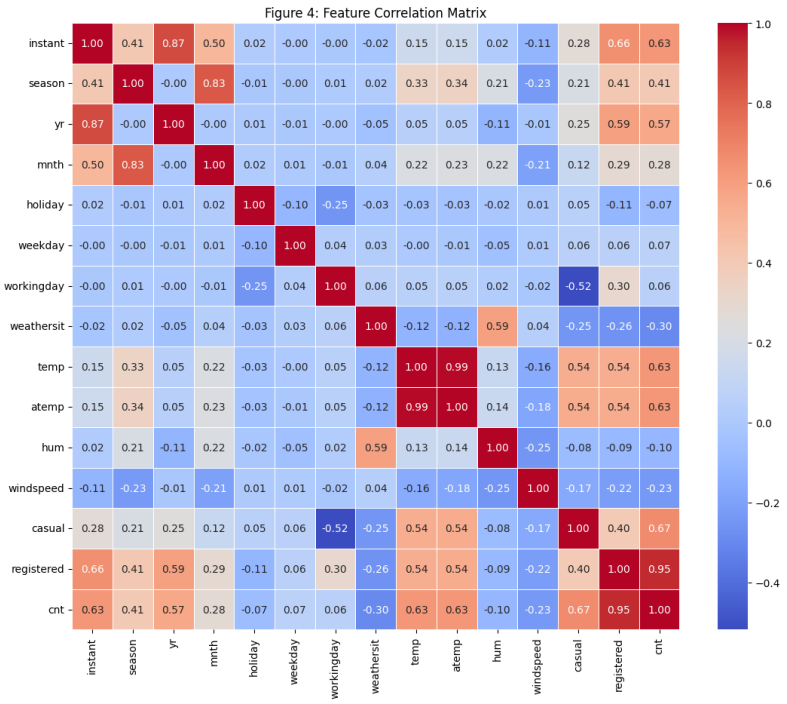

Figure 4: Heatmap showing feature correlations with 'cnt'.

9 Model Performance

The Random Forest model was evaluated on a 20% test split:

- **Mean Squared Error (MSE):** 458,446.95
- **R-Squared (R^2):** 0.8857 (88.6% accuracy)

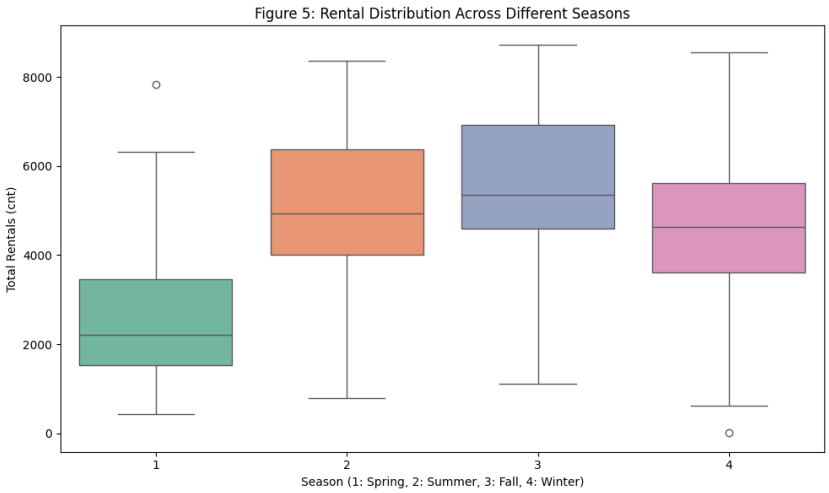

Figure 5: Rental distribution across different seasons.

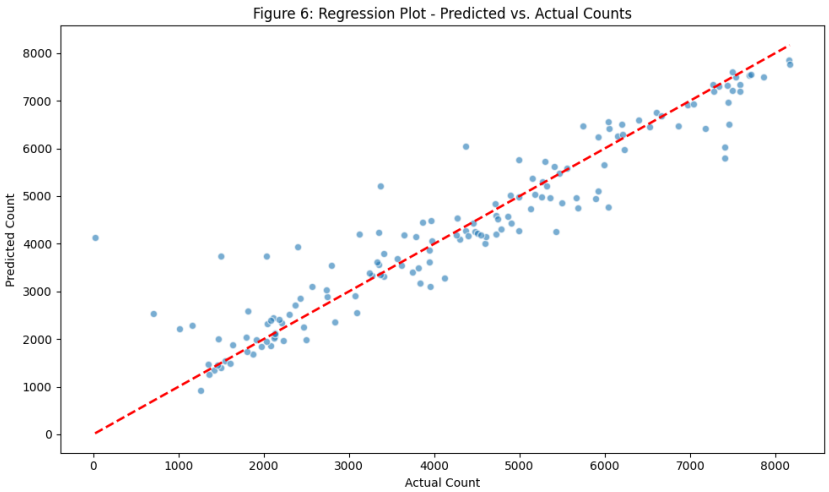

Figure 6: Regression plot: Predicted vs. Actual counts.

10 Conclusion

Temperature (`temp`) shows the strongest positive correlation with bike usage. The Random Forest model proved highly effective, explaining approximately 88.6% of the variance in the daily rental counts.